

Folio

Arquitetura de Camadas

Versão 1.0 • Clean Architecture + TypeScript + Node.js

1. Introdução

Este documento define os padrões de arquitetura de camadas do Folio — como o código do backend é organizado, quais responsabilidades cada camada tem, como elas se comunicam e quais regras nunca devem ser quebradas.

A arquitetura escolhida é uma adaptação da Clean Architecture de Robert C. Martin, ajustada para o contexto de um projeto Node.js + TypeScript + Express com Prisma. O objetivo não é seguir a teoria à risca — é garantir que o código seja legível, testável e fácil de evoluir conforme o produto cresce.

A pergunta que esta arquitetura responde é: quando um novo desenvolvedor (ou você mesmo, 6 meses depois) abre o projeto, ele consegue entender onde cada coisa vive e por quê?

2. O princípio central: direção das dependências

Toda a arquitetura do Folio segue uma única regra inviolável: as dependências apontam sempre para dentro. Camadas externas conhecem as internas. Camadas internas nunca conhecem as externas.

Na prática, isso significa:

- A lógica de negócio (Services) não sabe que existe Express, Prisma ou banco de dados.
- As entidades (Entities) não sabem que existe Service, Controller ou banco.
- Se você precisar trocar o banco de PostgreSQL para MySQL, só muda o Repository — o Service não toca.
- Se você precisar trocar Express por Fastify, só muda o Controller — o Service não toca.

Esse princípio é o que torna o código testável: você consegue testar a lógica de negócio sem subir banco, sem subir servidor, sem nenhuma dependência externa.

3. As quatro camadas

O backend do Folio é organizado em quatro camadas concêntricas, da mais interna para a mais externa:

Camada	Responsabilidade	Exemplos no Folio
Entities	Objetos de domínio — as regras mais fundamentais do negócio	Note, Habit, Task, DailyLog, User
Use Cases	Regras de aplicação — o que o sistema faz, um caso por vez	CreateNote, MarkHabit, MigrateUnfinishedTasks

Interface Adapters	Tradução entre mundo externo e use cases	Controllers, DTOs, Presenters, Mappers
Infrastructure	Detalhes externos — banco, email, framework, plugins	PrismaRepository, ExpressRouter, ResendMailer

3.1 Entities — o núcleo

Entities são as classes e tipos que representam os conceitos centrais do Folio. Elas contêm as regras de negócio mais fundamentais — aquelas que nunca mudam independente de framework, banco ou interface.

Uma Entity não importa nada externo. Ela é TypeScript puro.

```
// src/domain/entities/Note.ts
export class Note {
  constructor(
    public readonly id: string,
    public readonly userId: string,
    public title: string,
    public content: string,
    public tags: string[],
    public readonly createdAt: Date,
    public updatedAt: Date,
  ) {}

  update(title: string, content: string): void {
    if (!title.trim()) throw new Error('Title cannot be empty')
    this.title = title
    this.content = content
    this.updatedAt = new Date()
  }

  addTag(tag: string): void {
    if (!this.tags.includes(tag)) this.tags.push(tag)
  }
}
```

Regras das Entities:

- Sem imports de libs externas — apenas TypeScript puro.
- Contêm validações e regras que sempre valem — independente de contexto.
- Nunca fazem I/O (sem banco, sem rede, sem arquivo).
- São imutáveis por padrão — métodos que mudam estado são explícitos e nomeados.

3.2 Use Cases — a lógica de aplicação

Use Cases são o coração do sistema. Cada Use Case representa uma ação que o usuário pode realizar — e somente uma. Eles orquestram Entities e chamam Repositories (via interface), mas nunca sabem como o dado é persistido ou de onde veio a requisição.

```
// src/application/use-cases/notes/CreateNoteUseCase.ts
import { Note } from '@domain/entities/Note'
import { INoteRepository } from '@domain/repositories/INoteRepository'
import { randomUUID } from 'crypto'
```

```
interface CreateNoteInput {
  userId: string
  title: string
  content: string
  tags?: string[]
}

export class CreateNoteUseCase {
  constructor(private noteRepo: INoteRepository) {}

  async execute(input: CreateNoteInput): Promise<Note> {
    if (!input.title.trim()) {
      throw new Error('Note title is required')
    }

    const note = new Note(
      randomUUID(),
      input.userId,
      input.title,
      input.content,
      input.tags ?? [],
      new Date(),
      new Date(),
    )

    return this.noteRepo.save(note)
  }
}
```

Regras dos Use Cases:

- Um arquivo por Use Case — sem classes que fazem várias coisas.
- Recebem interfaces de repositório, nunca implementações concretas.
- Toda validação de regra de negócio fica aqui ou na Entity.
- Nunca importam Express, Prisma ou qualquer detalhe de infraestrutura.
- São facilmente testáveis com repositórios em memória (mocks).

3.3 Interface Adapters — a ponte

Esta camada traduz o mundo externo (HTTP, JSON, query params) para o formato que os Use Cases entendem, e vice-versa. É aqui que vivem os Controllers, os DTOs e os Mappers.

Controllers

O Controller recebe a requisição HTTP, valida o formato dos dados de entrada, chama o Use Case e devolve a resposta. Ele não tem lógica de negócio — só orquestração de entrada/saída.

```
// src/adapters/controllers/notes/CreateNoteController.ts
import { Request, Response } from 'express'
import { CreateNoteUseCase } from '@application/use-cases/notes/CreateNoteUseCase'
import { CreateNoteDTO } from '@adapters/dtos/notes/CreateNoteDTO'
import { NotePresenter } from '@adapters/presenters/NotePresenter'

export class CreateNoteController {
```

```

constructor(private useCase: CreateNoteUseCase) {}

async handle(req: Request, res: Response): Promise<void> {
  const dto = CreateNoteDTO.parse(req.body) // valida formato

  const note = await this.useCase.execute({
    userId: req.user.id,           // vem do middleware de auth
    title: dto.title,
    content: dto.content,
    tags: dto.tags,
  })

  res.status(201).json(NotePresenter.toHTTP(note))
}

```

DTOs — Data Transfer Objects

DTOs definem o formato esperado dos dados que chegam via HTTP. Eles validam shape e tipos — usando Zod — antes de qualquer lógica de negócio ser executada.

```

// src/adapters/dtos/notes/CreateNoteDTO.ts
import { z } from 'zod'

const schema = z.object({
  title: z.string().min(1).max(200),
  content: z.string().default(''),
  tags: z.array(z.string()).optional(),
})

export const CreateNoteDTO = {
  parse: (data: unknown) => schema.parse(data)
}

```

Presenters

Presenters formatam a saída — transformam a Entity em JSON limpo para a resposta HTTP. Aqui você decide o que expor e o que omitir.

```

// src/adapters/presenters/NotePresenter.ts
import { Note } from '@domain/entities/Note'

export const NotePresenter = {
  toHTTP: (note: Note) => ({
    id: note.id,
    title: note.title,
    content: note.content,
    tags: note.tags,
    createdAt: note.createdAt.toISOString(),
    updatedAt: note.updatedAt.toISOString(),
    // userId nunca exposto na resposta
  })
}

```

3.4 Infrastructure — os detalhes externos

A camada de infraestrutura contém tudo que é detalhe de implementação: o banco de dados, o cliente de email, o framework HTTP, o sistema de plugins. Esta é a única camada que pode importar Prisma, Express, Resend ou qualquer lib externa.

Repositories

O Repository implementa a interface definida no domínio. O Use Case conhece apenas a interface — nunca a implementação concreta.

```
// src/domain/repositories/INoteRepository.ts ← interface no domínio
import { Note } from '@domain/entities/Note'

export interface INoteRepository {
  save(note: Note): Promise<Note>
  findById(id: string): Promise<Note | null>
  findByUserId(userId: string): Promise<Note[]>
  update(note: Note): Promise<Note>
  delete(id: string): Promise<void>
}
```

```
// src/infrastructure/repositories/PrismaNoteRepository.ts ← implementação
import { PrismaClient } from '@prisma/client'
import { INoteRepository } from '@domain/repositories/INoteRepository'
import { Note } from '@domain/entities/Note'
import { NoteMapper } from '@infrastructure/mappers/NoteMapper'

export class PrismaNoteRepository implements INoteRepository {
  constructor(private prisma: PrismaClient) {}

  async save(note: Note): Promise<Note> {
    const record = await this.prisma.note.create({
      data: NoteMapper.toPrisma(note)
    })
    return NoteMapper.toDomain(record)
  }

  async findById(id: string): Promise<Note | null> {
    const record = await this.prisma.note.findUnique({ where: { id } })
    return record ? NoteMapper.toDomain(record) : null
  }
  // ... outros métodos
}
```

4. Estrutura de pastas completa

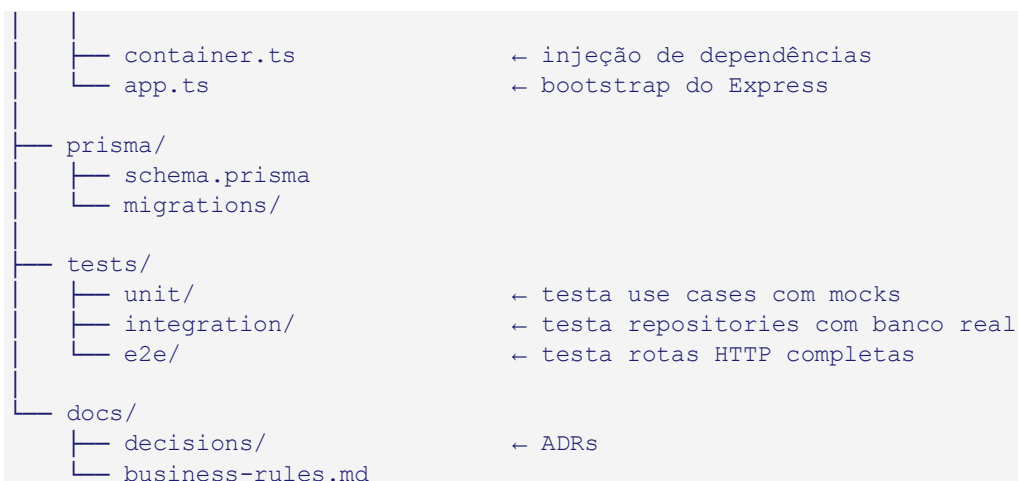
A estrutura de pastas reflete diretamente as quatro camadas. Cada pasta tem um papel claro — sem 'helpers', sem 'utils' genéricos, sem pastas que misturam responsabilidades.

```
folio-api/
├── src/
│   ├── domain/ ← ENTITIES + INTERFACES
│   │   ├── entities/
│   │   └── Note.ts
```

```

├── Habit.ts
├── Task.ts
├── DailyLog.ts
├── User.ts
├── repositories/           ← interfaces (contratos)
│   ├── INoteRepository.ts
│   ├── IHabitRepository.ts
│   ├── ITaskRepository.ts
│   └── IUserRepository.ts
├── errors/                 ← erros de domínio tipados
│   ├── NotFoundError.ts
│   └── UnauthorizedError.ts
├── application/           ← USE CASES
│   ├── use-cases/
│   │   ├── notes/
│   │   │   ├── CreateNoteUseCase.ts
│   │   │   ├── UpdateNoteUseCase.ts
│   │   │   └── DeleteNoteUseCase.ts
│   │   ├── habits/
│   │   │   ├── CreateHabitUseCase.ts
│   │   │   └── MarkHabitDoneUseCase.ts
│   │   ├── daily-log/
│   │   │   ├── AddEntryUseCase.ts
│   │   │   └── MigrateUnfinishedTasksUseCase.ts
│   │   └── auth/
│   │       ├── RegisterUseCase.ts
│   │       └── LoginUseCase.ts
├── adapters/              ← INTERFACE ADAPTERS
│   ├── controllers/
│   │   ├── notes/
│   │   ├── habits/
│   │   ├── daily-log/
│   │   └── auth/
│   ├── dtos/              ← validação de entrada (Zod)
│   │   ├── notes/
│   │   └── habits/
│   ├── presenters/        ← formatação de saída
│   │   ├── NotePresenter.ts
│   │   └── HabitPresenter.ts
│   └── middlewares/
│       ├── auth.middleware.ts
│       └── error.middleware.ts
├── infrastructure/        ← INFRAESTRUTURA
│   ├── repositories/      ← implementações Prisma
│   │   ├── PrismaNoteRepository.ts
│   │   └── PrismaHabitRepository.ts
│   ├── mappers/           ← Prisma record ↔ Entity
│   │   ├── NoteMapper.ts
│   │   └── HabitMapper.ts
│   ├── plugins/           ← sistema de plugins
│   │   ├── PluginRegistry.ts
│   │   └── FolioAPIImpl.ts
│   └── mail/
│       └── ResendMailer.ts
├── routes/                ← registro de rotas Express
│   ├── notes.routes.ts
│   ├── habits.routes.ts
│   └── index.ts

```



5. Injeção de dependências

A Clean Architecture exige que Use Cases recebam interfaces, não implementações. Isso é resolvido via injeção de dependências — um único arquivo que monta o grafo de dependências da aplicação.

O Folio não usa um framework de DI (como tsyringe ou inversify) no MVP — um container manual é suficiente e mais legível para quem está aprendendo.

```
// src/container.ts
import { PrismaClient } from '@prisma/client'
import { PrismaNoteRepository } from
  '@/infrastructure/repositories/PrismaNoteRepository'
import { PrismaHabitRepository } from
  '@/infrastructure/repositories/PrismaHabitRepository'
import { CreateNoteUseCase } from '@/application/use-
  cases/notes/CreateNoteUseCase'
import { CreateNoteController } from
  '@/adapters/controllers/notes/CreateNoteController'

const prisma = new PrismaClient()

// repositories
const noteRepo = new PrismaNoteRepository(prisma)
const habitRepo = new PrismaHabitRepository(prisma)

// use cases
const createNoteUseCase = new CreateNoteUseCase(noteRepo)

// controllers
export const createNoteController = new CreateNoteController(createNoteUseCase)
```

Essa abordagem tem uma vantagem direta nos testes: para testar o CreateNoteUseCase, você passa um repositório em memória no lugar do PrismaNoteRepository — sem banco, sem setup complexo.

6. Tratamento de erros

Erros no Folio são tipados — não são strings genéricas. Cada erro de domínio tem sua própria classe, o que permite ao middleware de erros tratá-los de forma diferente.

```
// src/domain/errors/NotFoundError.ts
export class NotFoundError extends Error {
  constructor(resource: string) {
    super(`${resource} not found`)
    this.name = 'NotFoundError'
  }
}

// src/domain/errors/UnauthorizedError.ts
export class UnauthorizedError extends Error {
  constructor() {
    super('Unauthorized')
    this.name = 'UnauthorizedError'
  }
}
```

```
// src/adapters/middlewares/error.middleware.ts
import { Request, Response, NextFunction } from 'express'
import { NotFoundError } from '@domain/errors/NotFoundError'
import { UnauthorizedError } from '@domain/errors/UnauthorizedError'
import { ZodError } from 'zod'

export function errorMiddleware(
  err: Error, req: Request, res: Response, next: NextFunction
) {
  if (err instanceof ZodError) {
    return res.status(400).json({ error: 'Validation error', details: err.errors })
  }
  if (err instanceof NotFoundError) {
    return res.status(404).json({ error: err.message })
  }
  if (err instanceof UnauthorizedError) {
    return res.status(401).json({ error: 'Unauthorized' })
  }
  console.error(err)
  res.status(500).json({ error: 'Internal server error' })
}
```

7. Testabilidade

A maior vantagem desta arquitetura é a testabilidade. Como os Use Cases dependem de interfaces, é trivial criar repositórios em memória para testes unitários — sem banco, sem lentidão, sem flakiness.

```
// tests/unit/notes/CreateNoteUseCase.test.ts
import { CreateNoteUseCase } from '@application/use-cases/notes/CreateNoteUseCase'
```

```
import { INoteRepository } from '@domain/repositories/INoteRepository'
import { Note } from '@domain/entities/Note'

// repositório em memória — sem banco de dados
class InMemoryNoteRepository implements INoteRepository {
  private notes: Note[] = []

  async save(note: Note): Promise<Note> {
    this.notes.push(note)
    return note
  }
  async findById(id: string): Promise<Note | null> {
    return this.notes.find(n => n.id === id) ?? null
  }
  async findByUserId(userId: string): Promise<Note[]> {
    return this.notes.filter(n => n.userId === userId)
  }
  async update(note: Note): Promise<Note> { return note }
  async delete(id: string): Promise<void> {
    this.notes = this.notes.filter(n => n.id !== id)
  }
}

describe('CreateNoteUseCase', () => {
  it('deve criar uma nota com título válido', async () => {
    const repo = new InMemoryNoteRepository()
    const useCase = new CreateNoteUseCase(repo)

    const note = await useCase.execute({
      userId: 'user-1',
      title: 'Minha primeira nota',
      content: 'Conteúdo da nota',
    })

    expect(note.title).toBe('Minha primeira nota')
    expect(note.userId).toBe('user-1')
  })

  it('deve lançar erro se título for vazio', async () => {
    const repo = new InMemoryNoteRepository()
    const useCase = new CreateNoteUseCase(repo)

    await expect(
      useCase.execute({ userId: 'user-1', title: '', content: '' })
    ).rejects.toThrow('Note title is required')
  })
})
```

8. Regras invioláveis

Estas regras não são sugestões — são restrições da arquitetura. Quebrá-las compromete os benefícios do design inteiro:

Regra	Explicação
-------	------------

Domain não importa infra	Entities e repositórios (interfaces) nunca importam Prisma, Express ou qualquer lib externa.
Use Cases não conhecem HTTP	Nenhum Use Case importa Request, Response ou qualquer conceito HTTP.
Controllers não têm lógica	Controllers só recebem, delegam e respondem. Zero if/else de negócio.
Repositories retornam Entities	Nunca retornam objetos Prisma raw — sempre mapeiam para Entity antes.
Um Use Case por arquivo	Sem classes tipo NoteService com 10 métodos. Um arquivo, uma responsabilidade.
Erros são tipados	Nunca throw new Error('string genérica'). Sempre use as classes de erro do domínio.
DTOs validam na entrada	Dados de entrada são validados com Zod no Controller, antes de chegar no Use Case.
Plugins usam FolioAPI	Plugins nunca acessam o banco diretamente — sempre pela API interna do core.

9. Fluxo completo de uma requisição

Para solidificar o modelo mental, veja o caminho completo de uma requisição POST /notes — criar uma nota — do início ao fim:

- 1. Express recebe POST /notes com { title, content, tags } no body.
 - O router delega para CreateNoteController.handle(req, res).
- 2. Controller valida o body com CreateNoteDTO.parse(req.body).
 - Se o formato for inválido, Zod lança ZodError → middleware retorna 400.
 - Se válido, extrai userId do req.user (preenchido pelo middleware de auth).
- 3. Controller chama this.useCase.execute({ userId, title, content, tags }).
 - Passa o controle para o Use Case — sem mais lógica no Controller.
- 4. CreateNoteUseCase valida a regra de negócio (título não pode ser vazio).
 - Se inválido, lança NotFoundError ou erro de domínio → middleware retorna código correto.
 - Se válido, instancia a Entity Note com os dados e um UUID gerado.
- 5. Use Case chama this.noteRepo.save(note).
 - noteRepo é a interface — o Use Case não sabe que por baixo é Prisma.
- 6. PrismaNoteRepository.save() converte a Entity para o formato Prisma via NoteMapper.
 - Executa prisma.note.create() e converte o resultado de volta para Entity.
- 7. Use Case recebe a Entity salva e a retorna para o Controller.
- 8. Controller chama NotePresenter.toHTTP(note) e retorna res.status(201).json(...).
 - O Presenter decide quais campos expor — userId nunca aparece na resposta.

10. Próximos passos

Com a arquitetura definida, o caminho natural é:

- Criar o schema.prisma com as entidades principais (User, Note, Habit, Task, DailyLog).
- Implementar o módulo de auth completo (Register + Login + middleware JWT).
- Desenvolver os Use Cases do Daily Log — o módulo mais central do produto.
- Escrever os testes unitários de cada Use Case com repositórios em memória.
- Configurar o pipeline de CI no GitHub Actions para rodar os testes a cada push.

Folio Arquitetura • v1.0 • Código que explica a si mesmo.